ADO.Net & Entity Framework

Mme. Fatima-Ezzahra AIT BENNACER
f.aitbennacer@emsi.ma

2026 - 2027

Plan du cours

1. Introduction à ORM
2. Entity framework
3. L'architecture d'Entity Framework
4. L'approche Database First : mise en place
5. Cas pratique

Introduction à ORM

ORM: Objet-Relationnel Mapping

- Programme informatique jouant le rôle du traducteur entre le modèle **relationnel** et le modèle **objet**.
- Permet d'interroger et manipuler les données à partir d'une base de données à l'aide d'un paradigme orienté objet.

- Deux composants dans les ORM :
  - **Entités:** Instanciation d'une classe (étudiant, enseignant, cours, projet, etc).
  - **Gestionnaire d'entités:** à utiliser pour persister les entités dans la base de données.

Entity Framework

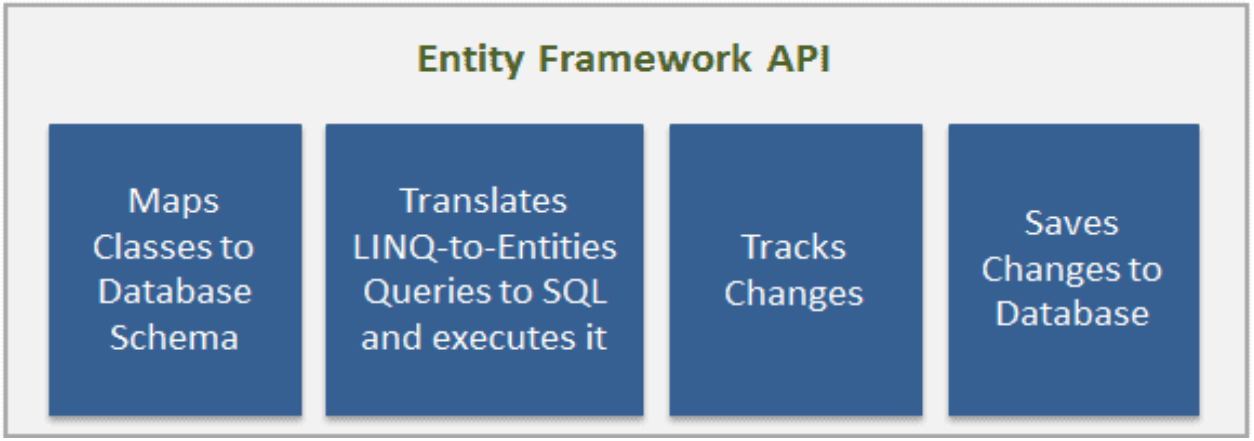
Entity Framework

- **Entity Framework** est un framework ORM open source pour les applications **.NET** prises en charge par **Microsoft** permettant de créer une couche d'accès aux données liées à une base de données relationnelle.
- Permettant aux développeurs de manipuler des données à l'aide d'objets de classes C# sans se concentrer sur les tables et colonnes d'une base de données (où ces données sont stockées).
- Permettant aux développeurs de créer et maintenir des applications orientées données avec moins de code, par rapport aux applications traditionnelles, grâce à **LinQ to entities.**

Le fonctionnement d'Entity Framework

- **Dans le cas d'Entity Framework**
  - Entité (Une classe qui correspond à une table de base de données)
  - Le gestionnaire d'entités : *LinQ to Entities*

- **Fonctionnement d'Entity Framework**
  - L'API Entity Framework a la possibilité de:

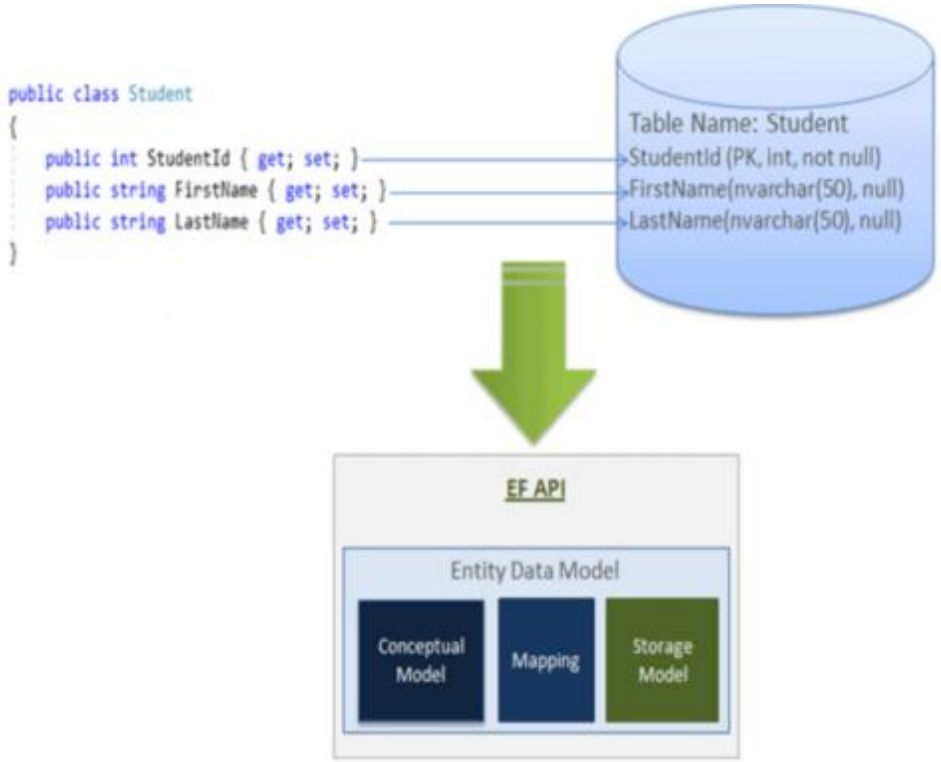


Le fonctionnement d'Entity Framework

- **EDM: Entity Data Model**
  - EDM est une représentation en mémoire de l'ensemble des métadonnées

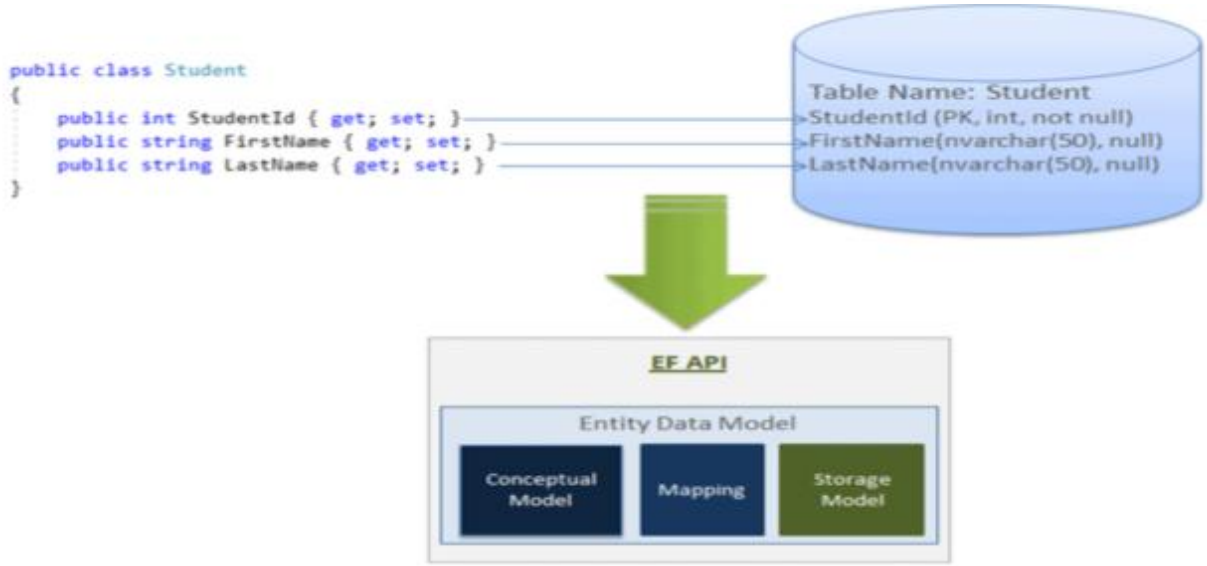
EDM consiste en trois parties:

- **Conceptuelle :** contient les classes du modèle et ses relations.
- **Stockage :** contient le modèle physique de la base : tables, vues, procédures stockées, les relations et les clés.
- **Mapping :** définit les mécanismes de passage du modèle conceptuel au stockage.



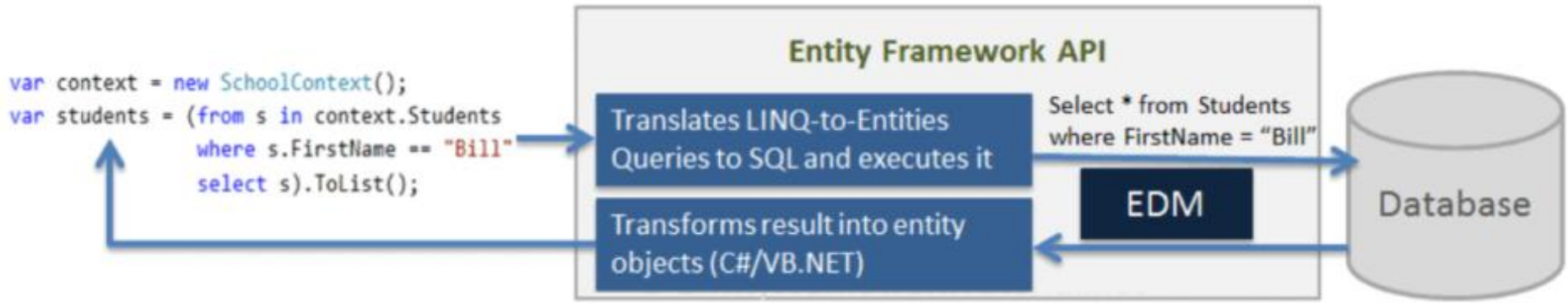
Maps Classes to Database scheme

- **A l'aide de EDM, EF peut:**
  - Effectuer des opérations **CRUD** (Create, Read, Update and Delete).
  - Créer des requêtes **SQL** à partir de requêtes **LINQ**, créer des commandes **INSERT, UPDATE et DELETE** et transformer le résultat de la base de données en objets d'entité.



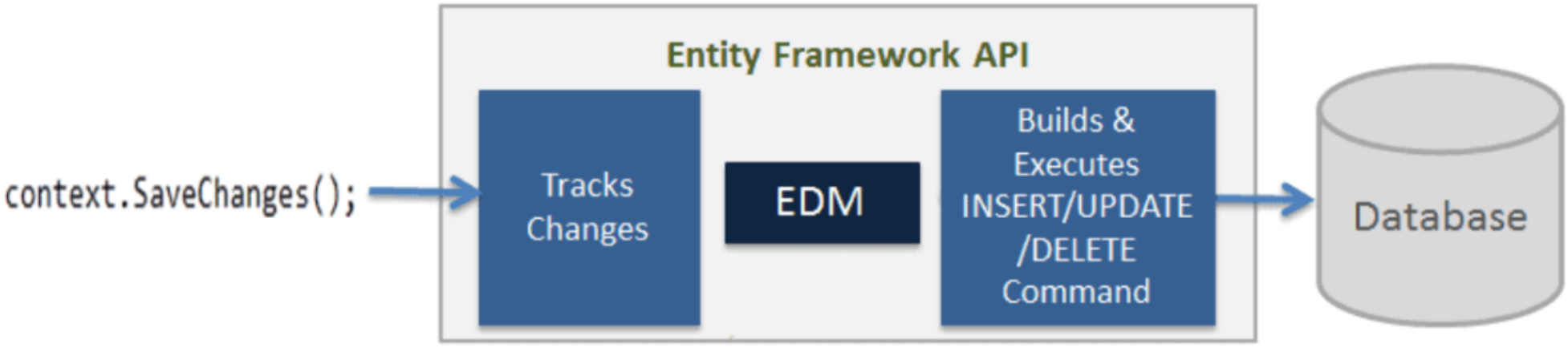
Translates LINQ-to-Entities Queries to SQL

- **À l'aide d'EDM, l'API EF:**
  - Traduit les requêtes **LINQ** en requêtes **SQL** pour les bases de données relationnelles.
    - **LinQ to Entity** : Un langage de requête pour un modèle orienté objet.
    - Il retourne les entités définies dans le modèle conceptuel.
  - Reconvertit les résultats (requêtes SQL) en objets d'entité.



Tracks and save changes to database

- Lorsque la méthode **SaveChanges()** est appelée, l'API EF déduit les commandes INSERT, UPDATE et DELETE en fonction de l'état des entités.
- Le **ChangeTrack** suit les changements survenus sur les entités



L'architecture de Entity Framework

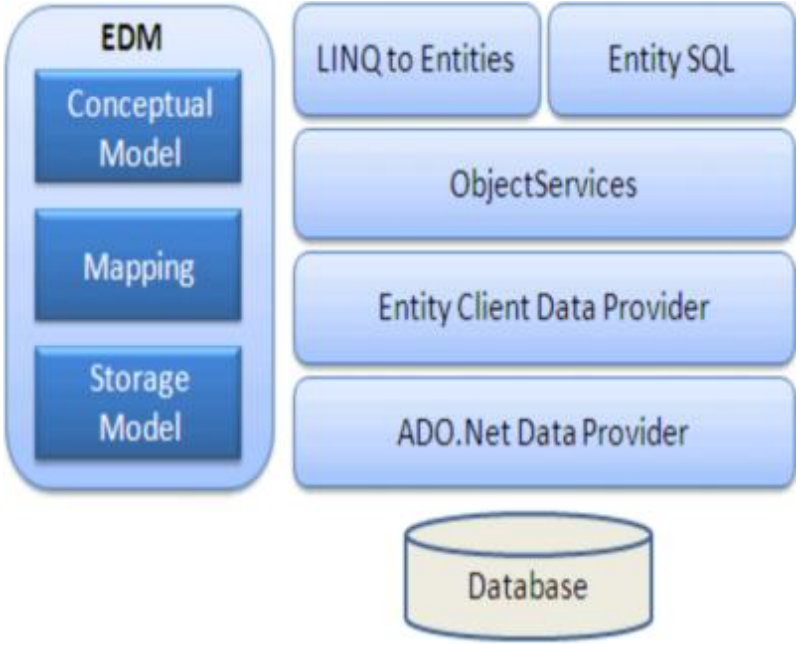
L'architecture de Entity Framework

EDM and LinQ to Entities

- **Object Service :** est le point d'entrée pour l'accès aux données d'une base de données. Il convertit les résultats SQL en objets C#

- **Entity Client Data Provider :** la tâche principale est de transformer une expression de LinQ à une requête SQL.

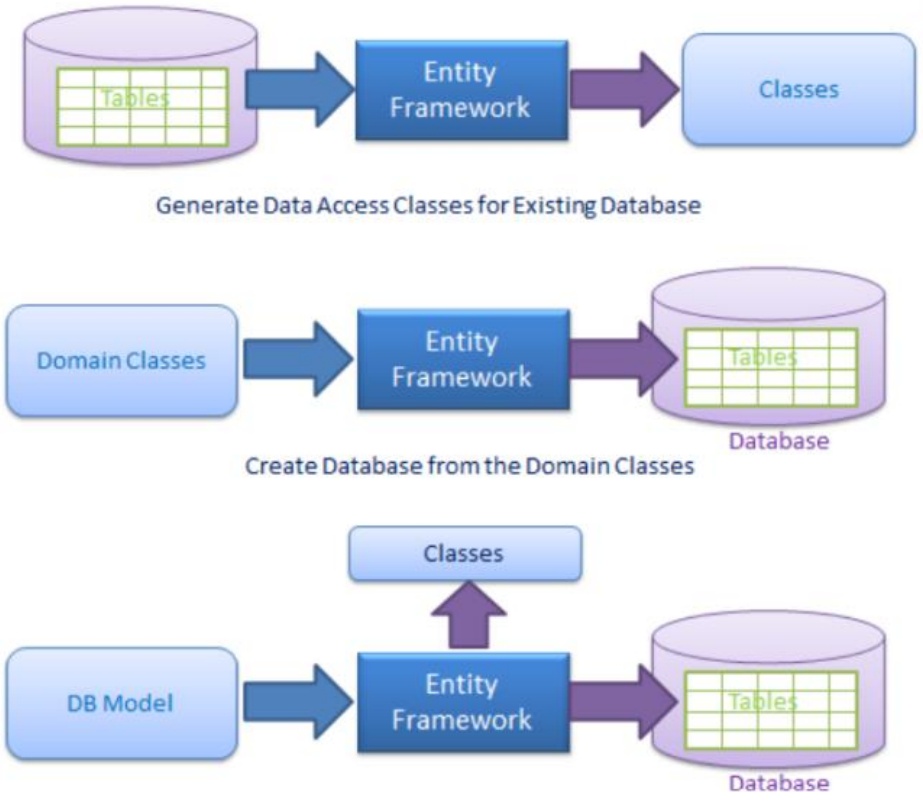
- **ADO.Net Data Provider :** Gère la communication technique avec la BD



Les différentes approches EF

Trois approches:

1. **Database First :** On crée la base de données (où on a une base de données qui existe déjà) et Entity Framework génère nos entités à partir de cette base de données.
2. **Code First :** On crée les entités puis et Entity Framework génère la base de données.
3. **Model First :** On crée notre modèle (de classe) et Entity Framework génère la base de données et les entités correspondantes.



Database First : mise en place

Database First

Database First :

- **Visual Studio** avec la charge de travail **Développement web et ASP.NET** et **.NET Framework 4.8**.
- **SQL Server** : édition **Developer** ou **Express**.
- **SSMS** (SQL Server Management Studio) : outil client pour créer la base et vérifier les données.
- Nom du serveur : `.` ou `localhost` (Developer), `.\SQLEXPRESS` (Express), `(localdb)\MSSQLLocalDB` (LocalDB).

Database First

Database First :

```
CREATE DATABASE BD_VENTE;
GO
USE BD_VENTE;
GO
CREATE TABLE PRODUITS (
    ID INT PRIMARY KEY IDENTITY(1,1),
    NOM VARCHAR(100) NOT NULL,
    DESCRIPTION VARCHAR(255),
    PRIX DECIMAL(10,2) NOT NULL,
    QUANTITE INT NOT NULL,
    DATE_AJOUT DATE DEFAULT GETDATE(),
    DISPONIBLE BIT DEFAULT 1
);
```

Database First

- **Insérer Les champs:**

```
INSERT INTO PRODUITS (NOM, DESCRIPTION, PRIX, QUANTITE)
VALUES
('Ordinateur Portable', 'PC 15 pouces, i5, 8Go RAM', 799.99, 12),
('Souris Optique', 'Souris USB avec capteur optique', 19.99, 50);

SELECT * FROM PRODUITS;
```

	ID	NOM	DESCRIPTION	PRIX	QUANTITE	DATE_AJOUT	DISPONIBLE
1	1	Ordinateur Portable	PC 15 pouces, i5, 8Go RAM	799.99	12	2025-11-02	1
2	2	Souris Optique	Souris USB avec capteur optique	19.99	50	2025-11-02	1

Database First : Créer le projet ASP.NET MVC 5

- Créer un nouveau projet sous Visual Studio **Fichier → Nouveau → Projet**.
- Sélectionner **Application web ASP.NET (.NET Framework)** puis **Suivant**.
- Nom du projet : **App_ACHAT,** puis Framework : **.NET Framework 4.8**
- Sélectionner MVC puis Créer

Database First : Ajouter l'Entity Data Model

EF designer à partir de la base de données

- Clic droit sur **Models** > **Ajouter** > **Nouvel élément**.
- Sélectionner **ADO.NET Entity Data Model**, nom : `Model1`, puis **Ajouter**.

- Les quatre options disponibles:
  - Sélectionner **EF designer à partir de la base de données** pour le contenu du modèle et cliquer sur Ajouter.

Database First : étapes à suivre

Connecter le serveur SQL avec Visual Studio

- Faire un clic sur Nouvelle connexion et saisir le nom du serveur (Faire un clic sur Object explorer dans SSMS).
- Sélectionner le nom de la base de données BD_VENTE.

Choix des objets (tables)

- Sélectionner les objets (les tables) que vous voulez inclure dans votre modèle (PRODUIT).
- Cocher Mettre au pluriel ou au singulier les noms des objets générés et cliquer sur Terminer

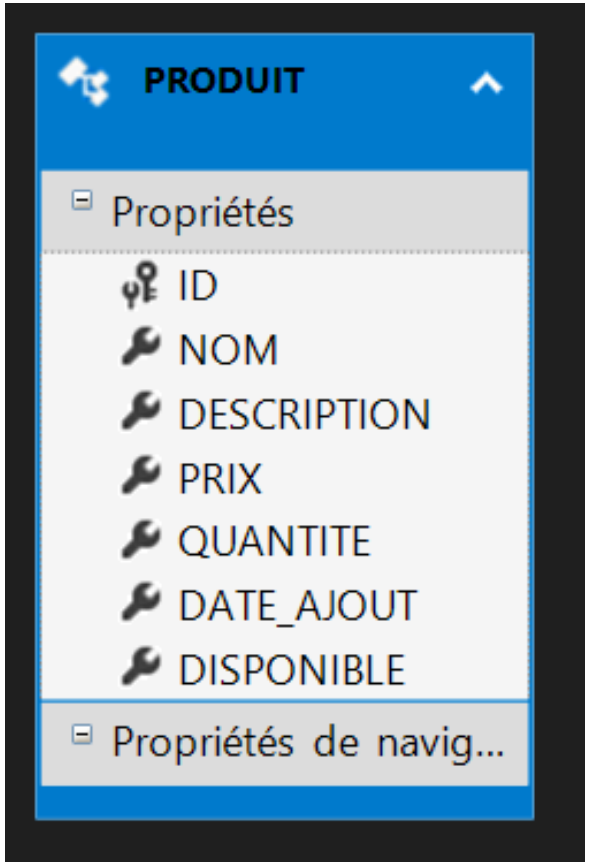
Database First : Les fichiers générés

Fichier	Rôle
Model1.edmx	Diagramme du modèle (EDM : conceptuel, stockage, mapping)
Model1.Context.cs	Classe BD_VENTEEntities : le **contexte** (hérite de DbContext)
PRODUIT.cs	Classe de l'**entité** correspondant à la table PRODUITS
Web.config	Chaîne de connexion BD_VENTEEntities

Ces fichiers sont **régénérés** à chaque mise à jour du modèle

Database First : Model1.edmx

Fichier Model1.edmx



DbContext et DbSet

La classe de contexte : Model1.Context.cs

```
public partial class BD_VENTEEntities : DbContext
{
    public BD_VENTEEntities()
        : base("name=BD_VENTEEntities")
    { }

    public virtual DbSet<PRODUIT> PRODUITS { get; set; }
}
```

Explication

- Une classe qui hérite de la classe **DbContext** est appelée classe de contexte dans le framework d'entité.
- La classe BD_VENTEEntities est une classe C# héritant de la classe DbContext.
- Le constructeur utilise la chaîne de connexion de **Web.config**.

Database First : DbContext

- **DbContext** est la classe principale responsable de l'interaction avec la base de données. Il est responsable de plusieurs activités:

  - **Querying:** convertit les requêtes LINQ-to-Entities en requêtes SQL et les envoie à la base de données.
  - **Change Tracking :** assure le suivi des modifications apportées aux entités après une requête à partir de la base de données.
  - **Persisting Data :** effectue les opérations d'insertion, de mise à jour et de suppression dans la base de données, en fonction des états de l'entité.
  - **Object Materialization :** convertit les données brutes de la base de données en objets d'entité.

Database First : DbSet

- La classe de contexte (BD_VENTEentities) inclut l'ensemble d'entités de type **DbSet<TEntity>**.
- **DbSet** est considérée comme une propriété de la classe de contexte et liée à une table de la base de données.
- **DBSet** contient un ensemble de méthodes permettant d'effectuer les opérations CRUD nécessaires.

- DbContext correspond à votre base de données (ou à une collection de tables et de vues dans votre base de données).
- DbSet correspond à une table ou une vue dans votre base de données.

Database First : PRODUIT.cs

PRODUIT.cs

```
public partial class PRODUIT
    {
        public int ID { get; set; }
        public string NOM { get; set; }
        public string DESCRIPTION { get; set; }
        public decimal PRIX { get; set; }
        public int QUANTITE { get; set; }
        public Nullable<System.DateTime> DATE_AJOUT { get; set;
}
        public bool DISPONIBLE { get; set; }
    }
```

- Les attributs de la classe PRODUIT sont les mêmes que ceux de la table PRODUITS.

Database First : Création du contrôleur

ProduitsController.cs

- **Générer la solution** (Ctrl+Maj+B) : sinon `PRODUIT` n'apparaît pas dans la liste.
- Clic droit sur **Controllers** > **Ajouter** > **Contrôleur**.
- Choisir **Contrôleur MVC 5 avec vues utilisant Entity Framework**, puis **Ajouter**.
- Classe de modèle : `PRODUIT (App_ACHAT.Models)`, classe de contexte : `BD_VENTEEntities (App_ACHAT.Models)`.
- Nom du contrôleur : `ProduitsController` ; **Ajouter**.

Database First : Création du contrôleur

Action	Méthode HTTP	Rôle
`Index()`	GET	Liste des produits
`Details(id)`	GET	Détail d'un produit
`Create()`	GET	Formulaire vide
`Create(produit)`	POST	Enregistre le nouveau produit
`Edit(id)`	GET	Formulaire pré-rempli
`Edit(produit)`	POST	Enregistre la modification
`Delete(id)`	GET	Page de confirmation
`DeleteConfirmed(id)`	POST	Supprime le produit

Vues dans `Views/Produits/` : `Index`, `Details`, `Create`, `Edit`, `Delete` (`.cshtml`).

ProduitsController: les opérations CRUD

Instanciation

```
private BD_VENTEEntities db = new BD_VENTEEntities();
// création d'une instance de la classe BD_VENTEEntities
```

Lire (Read)

```
public ActionResult Index()
 {
     return View(db.PRODUITS.ToList());
 }
```

- Récupère la liste de tous les produits depuis la base de données.

ProduitsController: les opérations CRUD

Créer (Create) POST

```
public ActionResult Create([Bind(Include =
"ID,NOM,DESCRIPTION,PRIX,QUANTITE,DATE_AJOUT,DISPONIBLE")] PRODUIT pRODUIT)
 {     if (ModelState.IsValid)     {
         db.PRODUITS.Add(pRODUIT);
         db.SaveChanges();
         return RedirectToAction("Index");     }
     return View(pRODUIT);  }
```

- **Bind(Include=…)** : les variables dont leurs valeurs vont être récupérées à partir du formulaire
- **if (ModelState.IsValid)**: validation côté serveur
- **db.PRODUITS.Add(pRODUIT):** ajout d'une nouvelle entité
- **db.SaveChanges():** Execute la commande **INSERT** pour l'entité avec l'état **Added** (Added state)
- **RedirectToAction("Index"):** passage de la méthode create à la méthode Index qui récupère la liste des produits ajoutés

ProduitsController: les opérations CRUD

Mettre à jour (Update):

```
public ActionResult Edit([Bind(Include =
"ID,NOM,DESCRIPTION,PRIX,QUANTITE,DATE_AJOUT,DISPONIBLE")]
PRODUIT pRODUIT)
{
    if (ModelState.IsValid)
    {   db.Entry(pRODUIT).State = EntityState.Modified;
        db.SaveChanges();
        return RedirectToAction("Index");    }
    return View(pRODUIT); }
```

- **db.Entry(pRODUIT).State = EntityState.Modified**: modification de l'entité pPRODUIT
- **db.SaveChanges();** Execute la commande **UPDATE** pour l'entité avec l'état Modified (Modifie l'url state)

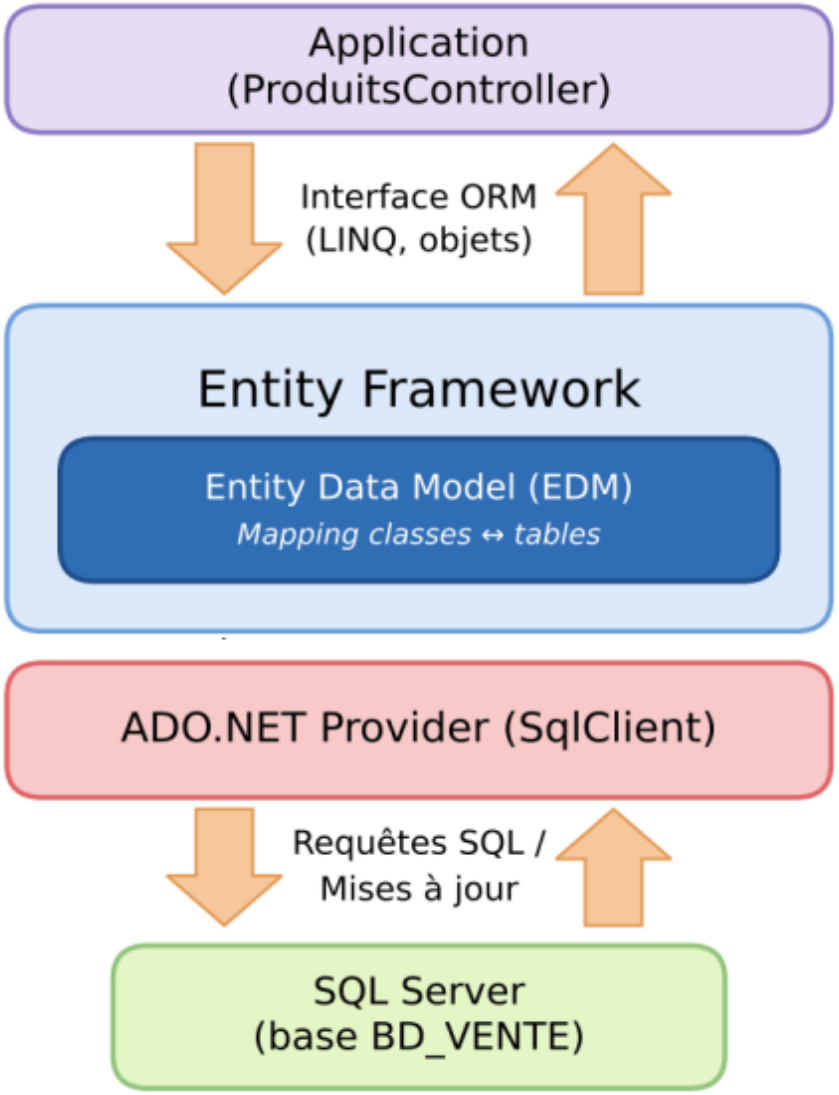
ProduitsController: les opérations CRUD

Supprimer (Delete):

```
public ActionResult DeleteConfirmed(int id)
{
    PRODUIT pRODUIT = db.PRODUITS.Find(id);
    db.PRODUITS.Remove(pRODUIT);
    db.SaveChanges();
    return RedirectToAction("Index");
}
```

- **PRODUIT pRODUIT = db.PRODUITS.Find(id);** Récupération de l'entité à partir la variable id dont la valeur est récupérée de l'url
- **db.PRODUITS.Remove(pRODUIT);** suppression de l'entité
- **db.SaveChanges():** Execute la commande **DELETE** pour l'entité avec l'état DELETED (DELETED state)

Où se place ADO.NET ?



Couche	Rôle
Application	Votre contrôleur : utilise LINQ et des objets C#.
Entity Framework	ORM : traduit LINQ en SQL, crée les objets, suit les modifications.
EDM	Décrit le mapping entre classes et tables.
ADO.NET Provider	Ouvre la connexion et exécute les commandes SQL.
SQL Server	Stocke les données (base BD_VENTE).

TP N°5

Relations entre les entités dans Entity Framework

Relations entre les entités dans Entity Framework

- Dans Entity Framework, une entité peut être associée à d'autres entités par le biais d'une association ou d'une relation.
- Chaque relation contient deux extrémités qui décrivent le type d'entité et la multiplicité du type (un, zéro-ou-un ou plusieurs) pour les deux entités de cette relation.

Les propriétés de navigation permettent de parcourir une association entre deux types d'entités. Chaque objet peut avoir une propriété de navigation pour chaque relation à laquelle il participe. Les propriétés de navigation vous permettent de parcourir et de gérer les relations dans les deux directions en retournant

- **Un objet de référence (si la multiplicité est UN ou zéro-ou-un)**
- **Une collection (si la multiplicité est plusieurs)**

Reference Navigation Property vs Collection Navigation Property

- Si une entité inclut une propriété d'un autre type d'entité, elle est appelée propriété de navigation de référence ou **Reference Navigation Property**. Il pointe vers une seule entité et représente la multiplicité d'un (1) dans les relations d'entité.

- Si une entité comprend une propriété de collection générique d'un type d'entité, elle est appelée propriété de navigation de collection **Collection Navigation Property**. Il représente la multiplicité de plusieurs (*).

Reference Navigation Property vs Collection Navigation Property

- Soit la base de données (BD_VENTE) définie par le modèle relationnel suivant :
  - **PRODUIT** (ID, NOM, DESCRIPTION, PRIX, QUANTITE, DATE_AJOUT, DISPONIBLE)
  - **CLIENT** (ID_CLIENT, NOM, PRENOM, EMAIL, VILLE)
  - **CLIENTAdress**(ID_CLIENT, ADRESSE1, STATE)
  - **COMMANDE** (ID_COMMANDE, ID_CLIENT, DATE_COMMANDE)
  - **DETAIL_COMMANDE** (ID_DETAIL, ID_COMMANDE, ID_PRODUIT, QUANTITE)

Reference Navigation Property vs Collection Navigation Property

Table CLIENT

Chaque ligne décrit un client ; les colonnes décrivent successivement :

- ID_CLIENT : identifiant unique du client (clé primaire)
- NOM : nom du client
- PRENOM : prénom du client
- VILLE : ville de résidence

➢ Cette table est liée à **CLIENTAdress** et **COMMANDE** par clé étrangère.

Table CLIENTAdress

Chaque ligne décrit une adresse associée à un client :

- ID_CLIENT : identifiant du client (clé primaire et étrangère vers CLIENT)
- ADRESSE1 : adresse principale
- STATE : région ou état (ex. : Fès-Meknès, Souss-Massa...)

➢ Un client peut avoir une seule adresse dans cette version simplifiée.

Reference Navigation Property vs Collection Navigation Property

Table COMMANDE

Chaque ligne décrit une commande passée par un client :

- ID_COMMANDE : identifiant unique de la commande (clé primaire)
- ID_CLIENT : identifiant du client ayant passé la commande (clé étrangère vers CLIENT)
- DATECOM : date de la commande

➢ Permet de suivre les achats de chaque client dans le temps.

Table DETAIL_COMMANDE

Chaque ligne représente un détail d'une commande (ligne d'achat) :

- ID_DETAIL : identifiant de la ligne (clé primaire)
- ID_COMMANDE : identifiant de la commande (clé étrangère vers COMMANDE)
- ID_PRODUIT : identifiant du produit (clé étrangère vers PRODUIT)
- QUANTITE : nombre d'unités commandées

➢ Table d'association entre COMMANDE et PRODUIT (relation N à N)

Reference Navigation Property vs Collection Navigation Property

Création des tables

PRODUITS

	ID	NOM	DESCRIPTION	PRIX	QUANTITE	DATE_AJOUT	DISPONIBLE
1	1	Ordinateur Portable	PC 15 pouces, i5, 8Go RAM	7990.00	12	2025-11-02	1
2	2	Souris Optique	Souris USB avec capteur optique	19.99	50	2025-11-02	1
3	3	Imprimante HP	Imprimante couleur jet d'encre	1200.00	25	2025-11-08	1
4	4	Écran Dell 24"	Écran Full HD 24 pouces	1850.50	40	2025-11-08	1
5	5	Clavier Logitech	Clavier sans fil	350.00	75	2025-11-08	1

CLIENT

	NUMCLI	NOMCLI	VILLE	CATEGORIE	COMPTE
1	1	Ahmed Karim	Casablanca	Gold	Actif
2	2	Sara El Amrani	Rabat	Silver	Inactif
3	3	Yassine Bakkali	Fès	Bronze	Actif

CLIENTAdress

	NUMCLI	ADDRESS1	STATE
1	1	RUE Z A AV A	Casablanca-Settat
2	2	RUE B AV B	Rabat-Salé-Kénitra
3	3	RUE C AV C	Fès-Meknès

COMMANDE

	NUMCOM	NUMCLI	DATECOM
1	1	1	2023-09-01
2	2	2	2023-09-05
3	3	3	2023-09-10

DETAIL_COMMANDE

	ID_DETAIL	ID_COMMANDE	ID_PRODUIT	QUANTITE
1	3	1	1	2
2	4	1	3	1
3	5	2	2	5

Reference Navigation Property vs Collection Navigation Property

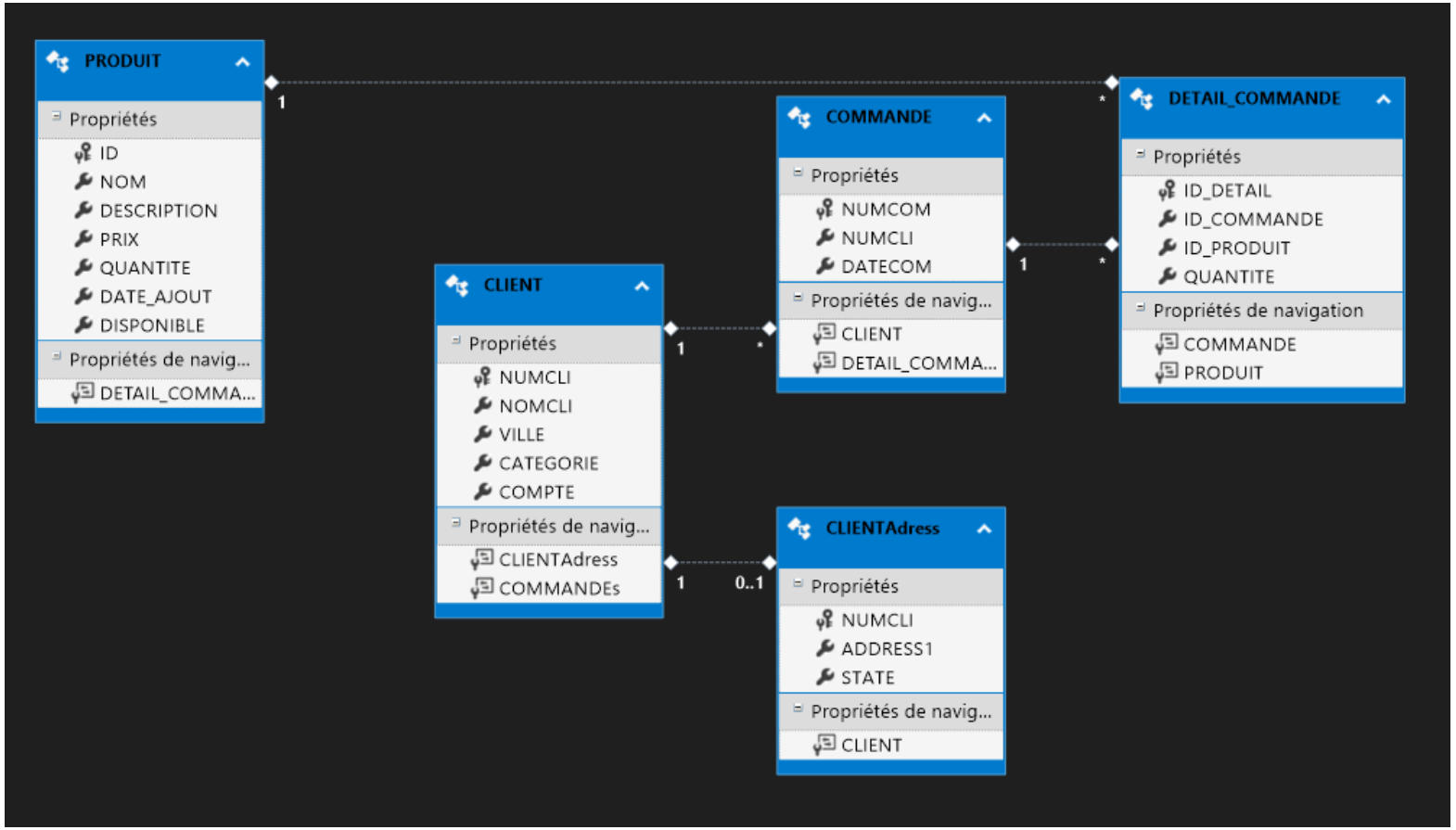
Model1.Context.cs

```
public partial class BD_VENTEEntities : DbContext
    {
public BD_VENTEEntities() : base("name=BD_VENTEEntities2")  {
}
        public virtual DbSet<PRODUIT> PRODUITS { get; set; }
        public virtual DbSet<CLIENT> CLIENTs { get; set; }
        public virtual DbSet<CLIENTAdress> CLIENTAdresses {
get; set; }
        public virtual DbSet<COMMANDE> COMMANDEs { get; set; }
        public virtual DbSet<DETAIL_COMMANDE> DETAIL_COMMANDE
{ get; set; }
    } }
```

- La classe BD_VENTEEntities est une classe C# héritant de la DbContext.
- La classe de contexte (BD_VENTEEntities) inclut les entités de type DbSet: CLIENTs, CLIENTAddresses, COMMANDEs, DETAILs et PRODUITs

Reference Navigation Property vs Collection Navigation Property

Model1.edmx



Classes générées : CLIENT.cs

```
public partial class CLIENT
{
    public CLIENT()
    {       this.COMMANDEs = new HashSet<COMMANDE>();   }

    public int NUMCLI { get; set; }
    public string NOMCLI { get; set; }
    public string VILLE { get; set; }
    public string CATEGORIE { get; set; }
    public string COMPTE { get; set; }
 // 1-1 : référence
    public virtual CLIENTAdress CLIENTAdress { get; set; }
// 1-N : collection
public virtual ICollection<COMMANDE> COMMANDEs { get; set; }
}
```

- Les colonnes deviennent des propriétés, les relations deviennent des propriétés de navigation.
- Le constructeur initialise la collection (HashSet) : elle n'est jamais null.

Reference Navigation Property vs Collection Navigation Property

Création du contrôleur CLIENTAddresses

- Faire un clic droit sur Controllers et aller dans Ajouter → Controleur.
- Sélectionner MVC 5 Controller avec vues, utilisant Entity Framework et cliquer sur Ajouter.
- Sélectionner CLIENTAddress (App Achat.Models) pour la classe de modèle et BD_VENTEEntities (App_ACHATS.Models) pour classe de contexte de données
- Saisir un nom pour le contrôleur (CLIENTAddressesController) et cliquer sur Ajouter

CLIENTAddressesController: les opérations CRUD

Instanciation

```
public partial class BD_VENTEEntities : DbContext
    {BD_VENTEEntities db = new BD ACHATEntities()
// création d'une instance de la classe BD_VENTEEntities
```

READ

```
public ActionResult Index() {
var cLIENTAddresses=db.CLIENTAddresses.Include(c=> c.CLIENT);
return View(cLIENTAddresses.ToList());
}
//Récupérer la liste des adresses ajoutées
//Récupérer la liste des clients (noms) ainsi que leurs
adresses.
// Include: Il s'agit de « Eager méthode », utilisée pour
récupérer un objet et les autres objets en relation dans une
seule requête.
```

CLIENTAddressesController: les opérations CRUD

CREATE

```
public ActionResult Create([Bind(Include =
"NUMCLI,ADDRESS1,STATE")] CLIENTAdress cLIENTAdress)
{
    if (ModelState.IsValid)
    {   db.CLIENTAdresses.Add(cLIENTAdress);
        db.SaveChanges();
        return RedirectToAction("Index");    }
    ViewBag.NUMCLI = new SelectList(db.CLIENTs, "NUMCLI",
"NOMCLI", cLIENTAdress.NUMCLI);
    return View(cLIENTAdress); }
```

- ViewBag.NUMCLI […] afin de récupérer la liste des numéros NUMCLI des clients créés. Les noms de ces clients sont affichés dans la liste déroulante (DropDownList) pour que l'utilisateur attribut une adresse à un client bien précis. (source, value, text, selectedValue)

CLIENTAddressesController: les opérations CRUD

UPDATE - GET

```
public ActionResult Edit(int? id)
{
    if (id == null)
    {        return new
HttpStatusCodeResult(HttpStatusCode.BadRequest);    }
    CLIENTAdress cLIENTAdress = db.CLIENTAdresses.Find(id);
    if (cLIENTAdress == null)
    {        return HttpNotFound();    }
    ViewBag.NUMCLI = new SelectList(db.CLIENTs, "NUMCLI",
"NOMCLI", cLIENTAdress.NUMCLI);
    return View(cLIENTAdress);  }
```

- CLIENTAddress cLIENTAddress = db.CLIENTAddresses.Find(id): Récupération de l'entité à partir la variable id dont la valeur est récupérée de l'url

CLIENTAddressesController: les opérations CRUD

UPDATE - POST

```
public ActionResult Edit([Bind(Include =
"NUMCLI,ADDRESS1,STATE")] CLIENTAdress cLIENTAdress)
 {     if (ModelState.IsValid)
     {   db.Entry(cLIENTAdress).State = EntityState.Modified;
         db.SaveChanges();
         return RedirectToAction("Index");     }
     ViewBag.NUMCLI = new SelectList(db.CLIENTs, "NUMCLI",
"NOMCLI", cLIENTAdress.NUMCLI);
     return View(cLIENTAdress);
 }
```

- db.Entry(cLIENTAddress).State = EntityState.Modified: modification de l'entité cLIENTAddress
- db.SaveChanges(); Execute la commande UPDATE pour l'entité (cLIENTAddress) avec l'état Modified (Modified state)

CLIENTAddressesController: les opérations CRUD

DELETE

```
public ActionResult DeleteConfirmed(int id)
 {
     CLIENTAdress cLIENTAdress = db.CLIENTAdresses.Find(id);
     db.CLIENTAdresses.Remove(cLIENTAdress);
     db.SaveChanges();
     return RedirectToAction("Index");
 }
```

- CLIENT cLIENT = db.CLIENTs.Find(id): Récupération de l'entité à partir la variable id dont la valeur est récupérée de l'url
- db.CLIENTAddresses.Remove(cLIENTAddress): suppression de l'entité cLIENTAddress
- db.SaveChanges(): Execute la commande DELETE pour l'entité (cLIENTAddress) avec l'état DELETED (DELETED state)

TP N°6